

Self-Evolved Imitation Learning in Simulated World

Yifan Ye [✉], Jun Cen [✉], Jing Chen [✉], and Zhihe Lu [✉], *Member, IEEE*

Abstract—Imitation learning has been a trend recently, yet training a generalist agent across multiple tasks still requires large-scale expert demonstrations, which are costly and labor-intensive to collect. To address the challenge of limited supervision, we propose Self-Evolved Imitation Learning (SEIL), a framework that progressively improves a few-shot model through simulator interactions. The model first attempts tasks in the simulator, from which successful trajectories are collected as new demonstrations for iterative refinement. To enhance the diversity of these demonstrations, SEIL employs dual-level augmentation: (i) Model-level, using an Exponential Moving Average (EMA) model to collaborate with the primary model, and (ii) Environment-level, introducing slight variations in initial object positions. We further introduce a lightweight selector that filters complementary and informative trajectories from the generated pool to ensure demonstration quality. These curated samples enable the model to achieve competitive performance with far fewer training examples. Extensive experiments on the LIBERO benchmark show that SEIL achieves a new state-of-the-art performance in few-shot imitation learning code is available at <https://github.com/Jasper-aaa/SEIL.git>.

Index Terms—Imitation learning, vision-language-action model, few-shot learning.

I. INTRODUCTION

IMITATION Learning (IL) [1], [2], [3], [4] has demonstrated remarkable success across a variety of tasks by leveraging large-scale expert demonstration datasets [5]. However, collecting such datasets is often time-consuming, labor-intensive [6], and in certain domains (*e.g.*, surgical practice) can even be impractical [7]. This motivates the study of IL under limited supervision, which is both a more practical and challenging setting. However, learning from only a few demonstrations typically results in a substantial performance drop. For instance,

Received 6 December 2025; accepted 29 March 2026. Date of publication 13 April 2026; date of current version 22 April 2026. This article was recommended for publication by Associate Editor D. Fridovich-Keil and Editor A. Faust upon evaluation of the reviewers' comments. This work was supported by the Research Computing Core Group Office of the Vice President for Research from Hamad Bin Khalifa University under the GPU compute grant Token Compression for Real-time Vision-Language-Action Models. (Corresponding authors: Jing Chen; Zhihe Lu.)

Yifan Ye and Zhihe Lu are with the College of Science and Engineering, Hamad Bin Khalifa University, Education City, Doha 34110, Qatar (e-mail: yyiye89305@hbku.edu.qa; zlu@hbku.edu.qa).

Jun Cen is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310058, China (e-mail: cenj@zju.edu.cn).

Jing Chen is with the School of Physics and Optoelectric Engineering, Guangdong University of Technology, Guangzhou 510006, China, and also with the Guangdong Provincial Key Laboratory of Sensing Physics and System Integration Applications, Guangdong University of Technology, Guangzhou 510006, China (e-mail: jchen125@gdut.edu.cn).

Code is available at <https://github.com/Jasper-aaa/SEIL.git>.

Digital Object Identifier 10.1109/LRA.2026.3683594

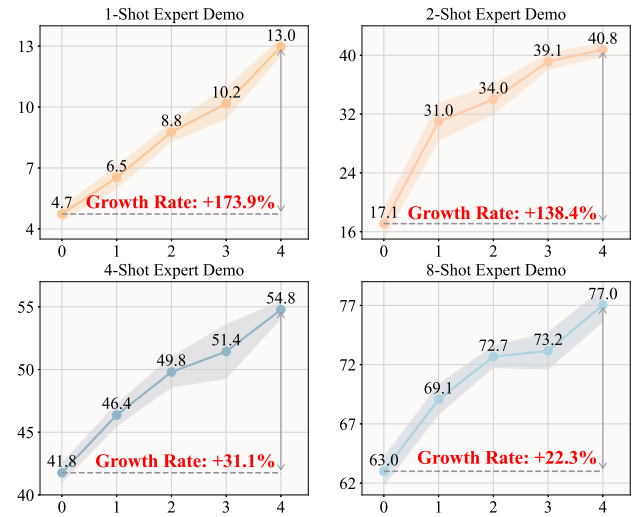


Fig. 1. Performance of the proposed SEIL on Libero Long. The x-axis denotes the number of evolutionary stages, while the y-axis represents the success rate on the benchmark at each stage. We report the performance growth rates throughout the evolution process.

in the one-shot setting, a train-from-scratch Diffusion Policy [2] achieves only 0.8% success rate, compared to 50.5% with full training.

Inspired by the empirical success of Reinforcement Learning (RL) [8], [9], [10], [11], [12], where agents explore through extensive trial-and-error within simulators, a natural solution is to leverage simulation environments to autonomously generate additional demonstrations for fine-tuning few-shot imitation policies. That is, a policy is initially trained on a limited set of expert demonstrations and then deployed in a simulator, where it interacts with the environment and records successful trajectories based on simple success/failure feedback. Once a sufficient number of demonstrations across tasks are collected, the policy is refined using standard imitation learning updates on the newly acquired data. However, this approach gives rise to two critical challenges: (i) How can we ensure sufficient diversity in the simulated demonstrations to support iterative learning? (ii) How can we identify and select the most informative samples to optimize model performance while maintaining computational efficiency?

In this paper, we investigate a few-shot imitation learning setting, address the aforementioned two key challenges and propose Self-Evolved Imitation Learning (SEIL), a novel framework that leverages simulator to enable policy self-evolution from limited expert demonstrations, often with significant improvements as shown in Fig. 1. SEIL is built upon two core components: dual-level augmentation, comprising model-level

and environment-level strategies, for generating diverse trajectories, and a sample selector tailored to identify and retain the most informative demonstrations for efficient policy refinement. Notably, SEIL supports multiple rounds of interaction with the simulator, enabling the policy to evolve progressively over time. Specifically, for model-level augmentation, SEIL introduces an auxiliary model to conduct additional rollouts alongside the primary model. To avoid incurring extra training overhead, particularly important in a multi-stage evolution setting where cost accumulates over iterations, this auxiliary policy is implemented as an Exponential Moving Average (EMA) of the main model. The EMA model maintains diversity in the policy space without the need for separate training, thereby supporting efficient and scalable demonstration generation. In addition to model-level augmentation, we introduce environment-level augmentation, which enables the policy to interact with the simulator under diverse initial conditions. This variation in starting states enhances environmental diversity, thereby enriching the recorded demonstrations and improving the robustness of policy learning.

After generating a pool of diverse demonstrations via dual-level augmentation, we introduce a lightweight selector to identify informative samples. The efficiency nature of this selector comes from taking as input only an initial image and a trajectory, rather than full video sequences. Trained on few-shot data via a trajectory classification task, the selector learns category-specific feature patterns. Once trained, the selector is frozen and applied to all recorded demonstrations. Samples with the lowest confidence, *i.e.*, those most distinct from expert demonstrations, are selected for policy training.

Our contributions are summarized as follows.

- We investigate a few-shot imitation learning setting and propose Self-Evolved Imitation Learning (SEIL), a novel framework that leverages simulated environments to enable policy self-evolution from limited expert demonstrations.
- We propose dual-level augmentations, *i.e.*, model-level and environment-level augmentation, to guarantee the diversity of demonstration generation.
- We propose a lightweight selector to identify the most informative demonstrations, enhancing both policy performance and training efficiency.
- Extensive experiments demonstrate that SEIL can effectively evolve weak few-shot trained models, achieving substantial improvements, *e.g.*, a 173.9% performance growth over the 1-shot baseline on LIBERO Long.

II. RELATED WORK

A. Learning From Offline Data

Imitation Learning (IL) [2] formulates policy learning as supervised behavior cloning from expert demonstrations, whereas Reinforcement Learning (RL) [13], [14], [15] relies on carefully designed reward functions. Recent works integrate IL and RL [16], [17], [18], using IL for initialization and RL for further reasoning and generalization. However, these approaches primarily emphasize the RL component [10], [11], [12]. In contrast, our SEIL framework centers on IL, removing reward

engineering through demonstration-based supervision and remaining effective under limited expert data, unlike conventional offline RL methods that depend on large static datasets and single-phase training [19], [20], [21].

B. Few-Shot Imitation Learning

Few-shot Imitation Learning (FIL) addresses settings with limited expert demonstrations [22]. Recent methods either leverage large foundation models for improved generalization [23], [24] or employ generative models to augment data diversity [25], [26]. However, foundation models introduce substantial computational overhead, and generative augmentation may yield unrealistic trajectories. In contrast, SEIL enables a weak few-shot policy to progressively improve without relying on heavy foundation models or synthetic data generation.

III. PRIMARIES

A. Problem Formulation

In this paper, we aim to enhance the performance of a multi-task imitation learning (IL) model trained on few-shot expert demonstrations by leveraging a simulator S . We consider an expert dataset $D_{\text{expert}} = \{(O_{1_i}, A_{1_i}), \dots, (O_{M_i}, A_{M_i})\}_{i=1}^{N_u}, T_u\}_{u=1}^L$, consisting of L tasks, where A_M denotes an action for task T and O_M is the corresponding robot observation. We focus on the low-data regime with $N_u \in \{1, 2, 4, 8\}$, consistent with standard few-shot settings [27], [28]. The simulator S , composed of task T and environment E , is used to generate the next observation O_{t+1} based on the provided action A_t at timestep t .

B. Vision-Language Action (VLA) Model

The VLA model can be defined as a function $\mathcal{F} : (O_u, T_u) \rightarrow A_u$, where O_u represents the observations, T_u represents the linguistic information, and A_u represents the actions for task T_u . The goal is to learn the mapping $\mathcal{F}$ that minimizes the action prediction error across tasks with demonstrations.

IV. PROPOSED METHODS

A. SEIL Framework

To improve an initial policy π_0 trained on few-shot expert demonstrations, we iteratively refine it through interaction with a simulator based on our proposed *dual-level augmentation* strategy. At iteration k , the current policy π_k is deployed in the simulator to generate a set of rollouts $\{\tau\}$. Based on binary task feedback, defined as completing task T within timestep budget t , we collect the successful trajectories, $\mathcal{R}_k = \{\tau \mid \text{Success}(\tau) = 1\}$, referred to as Recorded Demonstrations (RD). A trajectory selector S is then applied to filter informative samples, $\tilde{\mathcal{D}}_k = \{\tau \in \mathcal{R}_k \mid S(\tau) = 1\}$. The policy is retrained on the aggregated dataset $\mathcal{D}_{k+1} = \mathcal{D}_E \cup \tilde{\mathcal{D}}_k$, yielding the updated policy π_{k+1} . This iterative mapping $\pi_k \rightarrow \pi_{k+1}$ defines the train-record-select-train cycle of the Self-Evolved Imitation Learning framework. The sequence $\{\pi_k\}_{k=0}^K$ terminates at π_f when empirical performance saturates, formally when the success rate satisfies $p_{k+1} < p_k$.

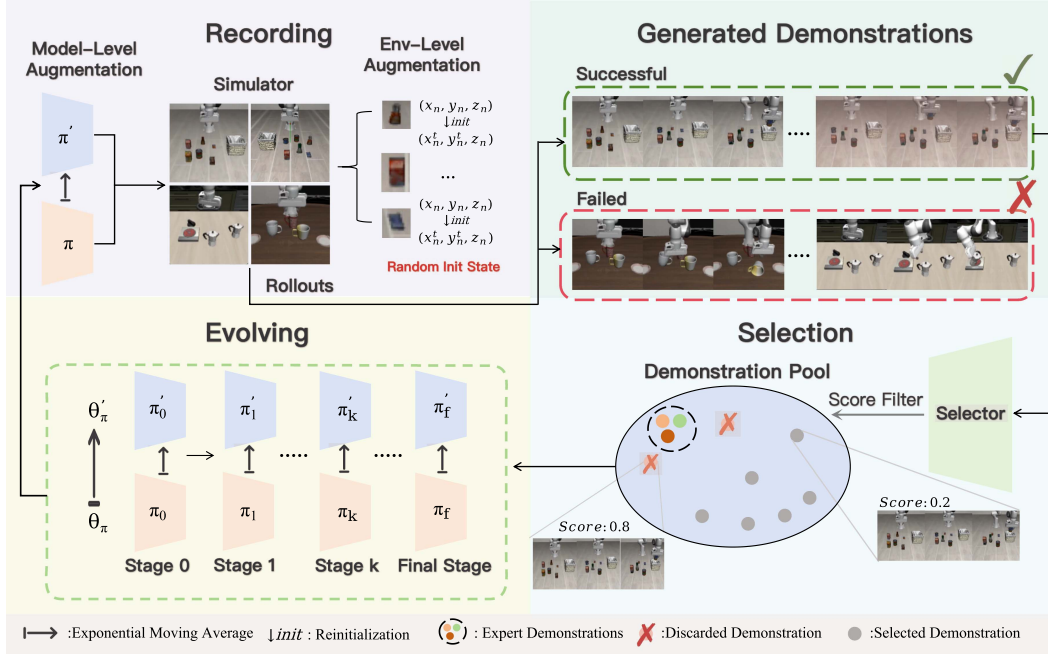


Fig. 2. Overall framework of our proposed SEIL Method. The framework begins by training a model π_0 using few-shot expert demonstrations. We then use the trained model to record demonstrations in the simulator. During this process, we apply model-level augmentation, where the trained policy π_0 interacts with the simulator alongside the EMA-enhanced model π'_0 , and environment-level augmentation, where the initial states are continuously varied to promote trajectory diversity. From simulator rollouts, we retain successful demonstrations and select the most informative using our proposed selector. The selected demonstrations incrementally expand the pool, allowing the model to evolve through iterative imitation learning updates until converging to the final policy π_f .

B. Dual-Level Augmentation

To enrich the diversity of RD, we propose a simple yet effective dual-level augmentation strategy. The first component, Environment-Level Augmentation, randomizes the simulator's initial states to expose the policy to varied environmental conditions. The second component, Model-Level Augmentation, leverages an auxiliary model with different parameters to generate complementary trajectories, enhancing behavioral diversity.

1) *Environment-Level Augmentation*: We first introduce Environment-Level Augmentation (E_{Aug}). As illustrated in the top-left panel of Fig. 2, prior to each interaction between the policy and the simulator, the position of each object is randomly perturbed. This transformation is defined as:

$$(x, y, z) \xrightarrow{init} (x_{new}, y_{new}, z), \quad (1)$$

where (x, y, z) denotes the default initial position of an object in the simulator, and $\xrightarrow{init}$ represents the random initialization function. A small offset is applied along the x and y axes to induce spatial variation while preserving vertical consistency.

2) *Model-Level Augmentation*: In Model-Level Augmentation (M_{Aug}), we introduce an auxiliary model to generate additional trajectories, with the key requirement that it incurs no extra training overhead. Since SEIL involves a multi-stage self-evolution process, training an additional policy at each stage would significantly increase the overall computational cost. To address this, we adopt an Exponential Moving Average (EMA) model, a lightweight, ensemble-style technique that avoids back-propagation while still producing diverse and stable rollouts for data augmentation.

Specifically, given the parameters at training step t of model π , denoted as θ_t , the EMA parameters θ_t^{EMA} are updated as:

$$\theta_t^{EMA} = \tau \cdot \theta_{t-1}^{EMA} + (1 - \tau) \cdot \theta_t, \quad (2)$$

where $\tau \in [0, 1)$ is the decay rate that controls the contribution of the most recent parameters. Intuitively, θ_t^{EMA} serves as a smoothed version of the online model parameters, thus providing better generalization and evaluation stability.

3) *Collaboration*: We deploy both the augmented model π' and the base model π in conjunction with the environment augmentation E_{Aug} , forming our Dual-Level Augmentation strategy. This combination effectively enhances the diversity of the generated trajectories. Implementing both E_{Aug} and M_{Aug} simultaneously is essential. If only E_{Aug} is used, the evolution process relies solely on the base model π , resulting in slow progress due to limited behavioral variability. On the other hand, using M_{Aug} alone introduces inconsistencies in input-action pairs, i.e., (O_t, A_t^{Base}) versus (O_t, A_t^{EMA}) , as the base and augmented models may produce divergent behaviors under identical observations. We further analyze the impact of these configurations in Section V-C1.

C. Trajectory Selector

1) *Overview*: We further propose a lightweight selector to identify and retain the most informative samples from the demonstration pool for policy evolution. Intuitively, effective selection requires the selector to capture both the distribution of RD and the action semantics within each trajectory. To this end, we train the selector using a task classification objective,

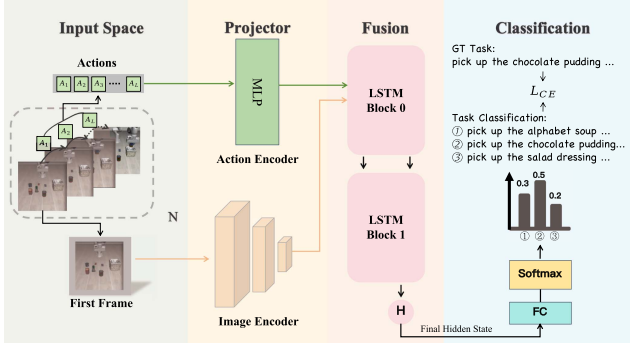


Fig. 3. Architecture of the proposed trajectory selector. We propose a lightweight trajectory selector composed of an image encoder and an action encoder, which process the first-frame image and the corresponding action sequence, respectively. The encoded features are fused through an LSTM, followed by an MLP that outputs a confidence score for each trajectory.

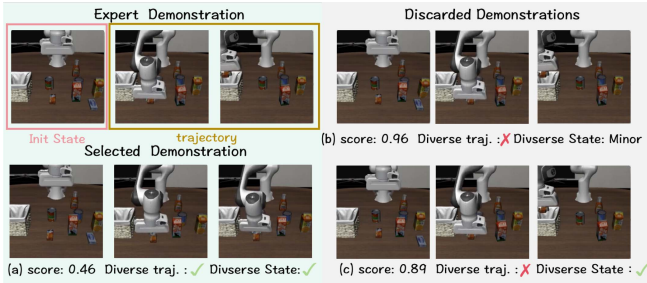


Fig. 4. We visualize the demonstrations selected by the selector along with their inference scores. The top-left panel shows the expert demonstration. Panels (a), (b), and (c) correspond to model rollouts, each annotated with the selector's predicted score.

enabling it to learn the underlying distribution of expert demonstrations. Once trained, the selector evaluates the RD and assigns confidence scores, allowing us to identify trajectories that are either representative or complementary to expert data, thereby enhancing diversity and effectiveness during model evolution.

2) *Architecture Design*: Our approach is inspired by the concept of world models in that we adopt a similar perspective to simplify a demonstration. Specifically, a world model can generate the next future state given an initial state (e.g., an image) and a corresponding action.

Motivated by this idea, we represent each demonstration using only its first frame and the corresponding action sequence, effectively decomposing the full trajectory into a compact representation. This design choice significantly reduces computational cost while preserving essential temporal and semantic information, thus enhancing the selector's capability to assess and differentiate demonstrations. Concretely, as illustrated in Fig. 3, given an input image $\mathbf{x} \in \mathbb{R}^{C \times H \times W}$ and an action sequence A , we employ a ResNet-18 [29] as the image encoder Φ_i and a multi-layer perceptron (MLP) as the action encoder Φ_a to extract feature representations. Here, the MLP is used solely to project the action representation from its original dimension to the hidden dimension of the LSTM. The encoding process is formally defined as:

$$\mathbf{f}_A = \Phi_a(A), \quad \mathbf{f}_x = \Phi_i(\mathbf{x}), \quad (3)$$

where $\mathbf{f}_A$ and $\mathbf{f}_x$ denote the action and image embeddings, respectively.

We then concatenate the two embeddings to obtain a fused representation $\mathbf{f}_t = [\mathbf{f}_x; \mathbf{f}_A]$, which is fed into a Long Short-Term Memory (LSTM) network to model temporal dependencies. The final hidden state $\mathbf{H}$ produced by the LSTM is passed through a classification MLP to generate task logits. The entire selector is optimized using the cross-entropy loss based on ground-truth task labels.

a) *Discussion*: A natural approach to demonstration task classification is to either (i) train a video model to classify full-length demonstration videos, or (ii) use a sequence model to classify trajectories based solely on the action sequences. While both methods can learn the distribution of the RD, they come with notable drawbacks. Video models, though expressive, introduce significant computational overhead and are susceptible to overfitting, especially in few-shot settings with limited expert demonstrations. Sequence-only models, on the other hand, ignore visual context and thus struggle to capture the semantics of recorded demonstrations. In contrast, our design strikes a balance between efficiency and representational power by using the first-frame image and the associated action sequence as input. This hybrid representation captures both visual and temporal cues while maintaining low computational cost, enabling effective task classification without the drawbacks of the above alternatives.

3) *Selection Scheme*: Once the selector is trained, we leverage its confidence scores over the RD to guide the selection of diverse training samples. We explore and compare three sampling strategies: (i) *Uniform sampling*: Randomly selecting demonstrations regardless of confidence, used as a baseline. (ii) *High-confidence sampling*: Selecting demonstrations with the highest confidence, i.e., those most similar to expert data. (iii) *Low-confidence sampling*: Selecting demonstrations with the lowest confidence, representing those most distinct from expert data.

While high-confidence sampling prioritizes expert-like behavior, it limits exposure to diverse trajectories and thus restricts policy generalization. In contrast, low-confidence sampling emphasizes diversity by selecting demonstrations farthest from the expert distribution, leading to richer learning signals and improved policy refinement. Our experiments show that this strategy consistently yields superior performance compared to the other alternatives.

V. EXPERIMENTS

A. Evaluation Benchmark

1) *Benchmark*: In our experiments, we adopt the LIBERO benchmark [6], which includes four sub-categories: LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO Long. LIBERO-Spatial evaluates spatial reasoning by requiring the robot to position a bowl based on its current location. LIBERO-Object tests object recognition and manipulation of distinct items. LIBERO-Goal focuses on procedural generalization by varying task goals while keeping the object set fixed. LIBERO Long consists of ten long-horizon tasks designed to assess the

TABLE I

COMPARISON ON LIBERO DATASET. FOR LIBERO LONG, WE CONDUCTED 5 EVOLUTION ROUNDS WITH A TOTAL OF 200 ROLLOUTS. FOR THE SPATIAL, OBJECT, AND GOAL BENCHMARKS, WE CONDUCTED 3 EVOLUTION ROUNDS WITH A TOTAL OF 120 ROLLOUTS. ALL VALUES ARE REPORTED AS SUCCESS RATE (%) OVER 50 EVALUATION ROLLOUTS. ALL RESULTS REPORTED ARE AVERAGED OVER 3 RANDOM SEEDS.

Dataset	Methods	1-shot	2-shot	4-shot	8-shot
LIBERO Long	DP	1.1 $\pm$ 0.4	7.3 $\pm$ 0.8	12.6 $\pm$ 1.0	18.9 $\pm$ 1.3
	ACT	0.6 $\pm$ 0.2	6.8 $\pm$ 0.7	15.4 $\pm$ 1.2	22.7 $\pm$ 1.5
	RT-1	0.3 $\pm$ 0.1	1.2 $\pm$ 0.3	3.8 $\pm$ 0.6	6.5 $\pm$ 0.9
	BAKU	4.7 $\pm$ 0.4	17.1 $\pm$ 0.8	41.8 $\pm$ 0.8	63.4 $\pm$ 1.6
	BAKU+SEIL	14.6 $\pm$ 1.7 ($\uparrow$ 9.9)	40.8 $\pm$ 1.1 ($\uparrow$ 23.7)	54.8 $\pm$ 1.6 ($\uparrow$ 13.0)	77.0 $\pm$ 2.1 ($\uparrow$ 13.6)
	MT-ACT	6.4 $\pm$ 0.8	27.2 $\pm$ 1.5	26.8 $\pm$ 3.2	30.4 $\pm$ 2.7
	MT-ACT+SEIL	17.2 $\pm$ 1.4 ($\uparrow$ 9.0)	49.5 $\pm$ 2.8 ($\uparrow$ 22.3)	51.0 $\pm$ 2.3 ($\uparrow$ 24.2)	66.7 $\pm$ 2.0 ($\uparrow$ 36.3)
LIBERO Spatial	DP	21.4 $\pm$ 1.2	30.6 $\pm$ 1.4	38.9 $\pm$ 1.6	44.8 $\pm$ 1.8
	ACT	10.2 $\pm$ 1.0	22.5 $\pm$ 1.3	50.7 $\pm$ 1.9	58.6 $\pm$ 2.0
	RT-1	2.1 $\pm$ 0.4	5.6 $\pm$ 0.7	12.4 $\pm$ 1.0	18.3 $\pm$ 1.3
	BAKU	42.9 $\pm$ 1.2	52.3 $\pm$ 1.2	75.8 $\pm$ 1.9	84.3 $\pm$ 1.9
	BAKU+SEIL	53.8 $\pm$ 1.5 ($\uparrow$ 10.9)	71.8 $\pm$ 1.6 ($\uparrow$ 19.5)	87.8 $\pm$ 1.7 ($\uparrow$ 12.0)	88.3 $\pm$ 1.4 ($\uparrow$ 4.0)
	MT-ACT	41.6 $\pm$ 1.3	39.1 $\pm$ 0.7	49.0 $\pm$ 1.5	78.6 $\pm$ 1.8
	MT-ACT+SEIL	50.1 $\pm$ 2.1 ($\uparrow$ 8.5)	65.6 $\pm$ 1.6 ($\uparrow$ 26.5)	72.0 $\pm$ 1.4 ($\uparrow$ 23.0)	87.2 $\pm$ 1.9 ($\uparrow$ 8.6)
LIBERO Object	DP	18.7 $\pm$ 1.1	26.9 $\pm$ 1.3	35.2 $\pm$ 1.5	41.3 $\pm$ 1.6
	ACT	12.4 $\pm$ 1.0	24.7 $\pm$ 1.2	47.5 $\pm$ 1.7	55.9 $\pm$ 1.9
	RT-1	3.6 $\pm$ 0.5	8.1 $\pm$ 0.8	15.6 $\pm$ 1.1	21.4 $\pm$ 1.3
	BAKU	39.8 $\pm$ 1.3	48.6 $\pm$ 1.4	71.4 $\pm$ 1.8	80.7 $\pm$ 1.9
	BAKU+SEIL	50.9 $\pm$ 1.6 ($\uparrow$ 11.1)	66.2 $\pm$ 1.7 ($\uparrow$ 17.6)	83.1 $\pm$ 1.5 ($\uparrow$ 11.7)	86.5 $\pm$ 1.6 ($\uparrow$ 5.8)
	MT-ACT	36.5 $\pm$ 1.2	38.7 $\pm$ 1.1	44.9 $\pm$ 1.6	74.3 $\pm$ 1.8
	MT-ACT+SEIL	47.8 $\pm$ 1.9 ($\uparrow$ 11.3)	61.4 $\pm$ 1.5 ($\uparrow$ 22.7)	69.7 $\pm$ 1.3 ($\uparrow$ 24.8)	84.9 $\pm$ 1.7 ($\uparrow$ 10.6)
LIBERO Goal	DP	16.9 $\pm$ 1.0	24.1 $\pm$ 1.2	32.8 $\pm$ 1.4	38.6 $\pm$ 1.5
	ACT	11.7 $\pm$ 0.9	23.4 $\pm$ 1.2	45.3 $\pm$ 1.8	53.2 $\pm$ 1.9
	RT-1	2.8 $\pm$ 0.4	6.3 $\pm$ 0.7	13.9 $\pm$ 1.0	19.6 $\pm$ 1.2
	BAKU	37.6 $\pm$ 1.2	46.8 $\pm$ 1.3	69.5 $\pm$ 1.7	78.9 $\pm$ 1.8
	BAKU+SEIL	48.4 $\pm$ 1.6 ($\uparrow$ 10.8)	63.9 $\pm$ 1.5 ($\uparrow$ 17.1)	81.2 $\pm$ 1.6 ($\uparrow$ 11.7)	84.7 $\pm$ 1.7 ($\uparrow$ 5.8)
	MT-ACT	34.9 $\pm$ 1.1	36.8 $\pm$ 1.0	42.7 $\pm$ 1.4	72.5 $\pm$ 1.7
	MT-ACT+SEIL	45.6 $\pm$ 1.8 ($\uparrow$ 10.7)	59.3 $\pm$ 1.4 ($\uparrow$ 22.5)	67.9 $\pm$ 1.3 ($\uparrow$ 25.2)	82.8 $\pm$ 1.6 ($\uparrow$ 10.3)
Average	DP	14.5 $\pm$ 0.9	22.2 $\pm$ 1.2	29.9 $\pm$ 1.4	35.9 $\pm$ 1.6
	ACT	8.7 $\pm$ 0.8	19.4 $\pm$ 1.1	39.7 $\pm$ 1.7	47.6 $\pm$ 1.8
	RT-1	2.2 $\pm$ 0.4	5.3 $\pm$ 0.6	11.4 $\pm$ 0.9	16.5 $\pm$ 1.2
	BAKU	31.3 $\pm$ 1.0	41.2 $\pm$ 1.2	64.6 $\pm$ 1.6	76.8 $\pm$ 1.8
	BAKU+SEIL	41.9 $\pm$ 1.6 ($\uparrow$ 10.6)	60.7 $\pm$ 1.5 ($\uparrow$ 19.5)	76.7 $\pm$ 1.6 ($\uparrow$ 12.1)	84.1 $\pm$ 1.7 ($\uparrow$ 7.3)
	MT-ACT	29.9 $\pm$ 1.1	35.5 $\pm$ 1.1	40.9 $\pm$ 1.7	63.9 $\pm$ 2.3
	MT-ACT+SEIL	40.2 $\pm$ 1.8 ($\uparrow$ 10.3)	59.0 $\pm$ 1.8 ($\uparrow$ 23.5)	65.2 $\pm$ 1.6 ($\uparrow$ 24.3)	80.4 $\pm$ 1.8 ($\uparrow$ 16.5)

robot’s ability to perform extended and temporally complex operations.

2) *Baselines*: We use two simple Vision-Language-Action model frameworks, MT-ACT [30] and BAKU [1], to validate the evolutionary feasibility of our proposed SEIL. We also report the few-shot performance of Diffusion Policy (DP) [2], Action Chunking Transformer (ACT) [31], and RT-1 [5] for comparison.

3) *Implementation Details*: In evolving, we allow both models to perform 30 successful rollouts in the simulator each round and use the selector to choose 20 of them to incrementally augment the evolving demonstration pool. Empirically, we set the EMA decay parameter τ to 0.999. For fairness, all the baselines and our method are trained from scratch. All ablation studies are conducted on the 8-shot LIBERO Long setting. For the selector, we use a pretrained ResNet-18 as the image encoder and stack two LSTM blocks, with an embedding hidden dimension of 256. The model is trained using Adam optimizer with a learning rate of 0.001 for 80 epochs.

4) *Metrics*: For the policy evaluation, we follow the standard LIBERO evaluation, each task is evaluated for 50 rollouts under benchmark initial states and we record the success rate (SR). For the selector, we report the classification accuracy on a validation set, which is constructed from 20 randomly sampled unseen expert demonstrations in LIBERO.

B. Main Results

From the averaged results in Table I, we first observe that all baseline methods perform poorly in the low-data regime, particularly under the 1-shot setting, indicating the inherent difficulty of generalization with extremely limited demonstrations. More importantly, SEIL consistently improves performance across all configurations and shot settings. Specifically, for BAKU, the average success rate increases from 31.3% to 41.9% in the 1-shot setting (+10.6%) and from 76.8% to 84.1% in the 8-shot setting (+7.3%). For MT-ACT, SEIL raises performance from 29.9% to 40.2% in the 1-shot setting (+10.3%) and from 63.9% to 80.4% in the 8-shot setting (+16.5%). These consistent gains across both strong (BAKU) and comparatively weaker (MT-ACT) backbones demonstrate that SEIL serves as an effective and model-agnostic performance amplifier, particularly in low-data scenarios.

C. Ablation Study

1) *Ablation Study on Different Components*. We analyze the impact of SEIL’s components in Table III, starting from the 8-shot baseline. We first discuss the success rate after four round evolution. Training with only baseline-recorded demonstrations achieves a higher SR (67.4%) than using both baseline and M_{Aug} -recorded data (67.2%). This drop is attributed to inconsistent trajectories from M_{Aug} under the same

TABLE II

ONE-ROUND EVOLUTION ACROSS DIFFERENT ROLLOUT POOLS. WE INITIALIZE EVOLUTION FROM 1-SHOT BAKU, 8-SHOT BAKU, AND 8-SHOT MT-ACT. FOR EACH MODEL, 20 ROLLOUTS ARE COLLECTED, AND 10 DEMONSTRATIONS ARE MANUALLY SELECTED BASED ON INITIAL-STATE AND TRAJECTORY DIVERSITY. SUCCESS RATES ARE REPORTED AFTER ONE EVOLVE ROUND USING ROLLOUT POOLS DERIVED FROM EACH INITIAL POLICY. *JOINT SELECTION* DENOTES MANUALLY SELECTING 10 ROLLOUTS FROM THE UNION OF THE THREE DEMONSTRATION POOLS. WE ADDITIONALLY COMPARE PROPOSED SELECTOR AGAINST HUMAN PREFERENCE. THE LAST ROW REPORTS THE OVERLAP RATIO BETWEEN THE 10 DEMONSTRATIONS SELECTED BY THE SELECTOR FROM A 20-ROLLOUT POOL AND THOSE CHOSEN MANUALLY.

Init Model/Rollout	1-shot BAKU	Δ	8-shot BAKU	Δ	8-shot MT-ACT	Δ	Average Δ
Base Performance	8.1	—	76.8	—	66.7	—	—
1-shot BAKU Rollout	12.5	+4.4	77.2	+0.4	69.3	+2.6	2.47
8-shot BAKU Rollout	34.4	+26.3	81.9	+5.1	73.5	+6.8	12.73
8-shot MT-ACT Rollout	22.5	+14.4	79.1	+2.3	71.6	+4.9	7.20
Joint Selection	35.7	+27.6	82.5	+5.7	74.1	+7.4	13.23
Selector Performance	40%	—	80%	—	70%	—	—

TABLE III

ABLATION STUDY ON DIFFERENT COMPONENTS. WE REPORT THE SUCCESS RATE AFTER ONE ROUND (SR_1) AND FOUR ROUNDS (SR_4) OF EVOLUTION. HERE, M_{Aug} AND E_{Aug} DENOTE MODEL-LEVEL AUGMENTATION AND ENVIRONMENT-LEVEL AUGMENTATION, RESPECTIVELY. “—” DENOTES NO E_{Aug} FOR THE FIRST ROUND.

Baseline	Evolve	M_{Aug}	E_{Aug}	Selector	SR_1	SR_4
✓					63.4%	63.4%
✓	✓				67.6%	67.4%
✓	✓	✓			67.3%	67.2%
✓	✓		✓		67.6%	73.4%
✓	✓	✓	✓		69.6%	74.6%
✓	✓	✓	✓	✓	69.8%	75.4%

environment E , where identical observations lead to different actions - undesirable for imitation learning. To be more specific, given an initial observation O_0 , the Base model and M_{Aug} produce different actions, A_0^{Base} and $A_0^{M_{Aug}}$, respectively. This discrepancy introduces inconsistency into the evolutionary process. Introducing E_{Aug} allows M_{Aug} to operate from varied initial states, mitigating this inconsistency and improving SR to 74.6%. Finally, applying our selector to retain only the most diverse and informative samples further boosts SR to **75.4%**. We further study the importance of E_{Aug} by analyzing the success rate after the first and fourth rounds. In the absence of E_{Aug} , the model is affected by the inconsistency between different supervisions, which results in a performance decline from 67.6% to 67.4%. With environment-level augmentation, the model achieves a 73.4% success rate at the fourth round, reflecting a 7% improvement compared to a single-round evolution (67.6%).

2) *Effects of Various Rollouts Combinations.* a) *Effects of EMA and Base model rollouts:* Fig. 5 reports the impact of various rollouts combinations generated by different models after a single round of evolution, providing further insights into the effect of proposed dual-level augmentation. We first analyze the individual rollout pools generated by the Base model (P1 + P2) and the EMA model (P3 + P4). Compared to Base model rollouts, which provide a 4.2% improvement over the baseline, evolving on EMA model rollouts further improves the few-shot Base model by 4.8% and the EMA model by 1.9%, indicating that EMA rollouts contain more informative demonstrations. Next, we combine both pools (P1 + P2 + P3 + P4) and evolve the model for one round. Despite the larger number of demonstrations for evolving, performance of Base model

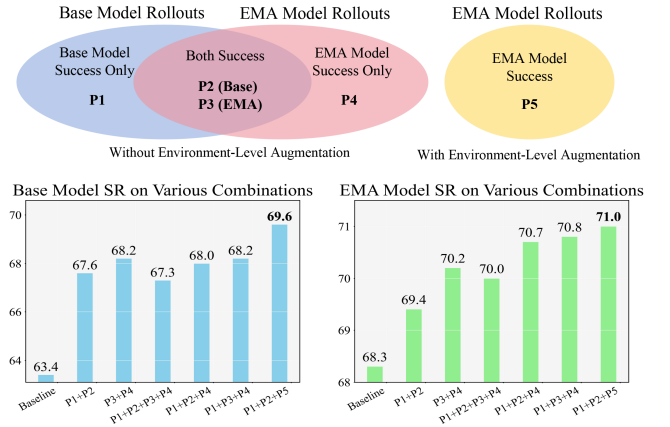


Fig. 5. Ablation study on evolution with different types of rollouts. We first record rollouts using the Base model and EMA model under identical environment settings. We divide the demonstration pool into three categories: Base Model success only (P1), EMA model success only (P4), and cases where both succeed (P2, P3). We further extend this setup by recording EMA model under varied environment settings, denoted as P5. We report the Success Rate (SR) of both Base and EMA model evolved on various rollouts combinations.

TABLE IV

ABLATION STUDY OF INJECTING NEAR SUCCESS (NS) AND FAIL DEMONSTRATIONS ACROSS DIFFERENT TRAINING ROUNDS. ALL Δ VALUES ARE MEASURED RELATIVE TO THE BASE MODEL (63.0). NEAR-SUCCESS DEMONSTRATIONS ARE GENERATED IN A CONTROLLED MANNER BY REPLAYING A SUCCESSFUL ROLLOUT AND INJECTING NOISE AT O_{n-K} , WHERE $k = 10$.

Round	Setting	SR	Δ
Base	Few-shot	63.0	—
Round 1	No Injection	69.1	+6.1
	Inject NS	69.0	+6.0
	Inject Fail	43.2	-19.8
Round 4	No Injection	77.0	+14.0
	Inject NS	74.5	+11.5
	Inject Fail	48.7	-14.3

TABLE V

EFFECT OF CANDIDATE ROLLOUT POOL SIZE ON FINAL PERFORMANCE. “K $\rightarrow$ 10” DENOTES GENERATING K ROLLOUTS AND SELECTING 10 FOR TRAINING, AND SIMILARLY FOR THE OTHER SETTINGS

Method	Baseline	10	20 $\rightarrow$ 10	50 $\rightarrow$ 10	100 $\rightarrow$ 10
SR (%)	63.4	65.1	65.2	66.3	67.2

TABLE VI
ABLATION OF NUMBER OF ROLLOUTS USED FOR TRAINING PER ROUND ON SIMULATOR

Rollouts\Round	Baseline	First Round	Second Round	Third Round	Fourth Round	Fifth Round
10	63.4%	65.1%	68.4%	68.6%	70.8%	72.0%
20	63.4%	67.8%	71.8%	72.4%	73.2%	74.2%
50	63.4%	70.6%	72.4%	71.4%	-	-
100	63.4%	72.1%	71.7%	-	-	-

declines to 67.3%, lower than evolving on either individual pool. To investigate this, we introduce two complementary groups: adding only EMA-successful demonstrations to the Base pool (P1 + P2 + P4) and vice versa (P1 + P3 + P4). Augmenting the Base pool with P4 increases the success rate to 68.0% (vs. 67.6% on P1 + P2), demonstrating that additional diverse demonstrations facilitate model evolution. Conversely, incorporating Base-successful rollouts into the EMA-generated pool (P1 + P3 + P4) results in an identical success rate of 68.2%. We investigate this by analyzing the number of supplementary demonstrations, which reveals that, since the EMA model outperforms the Base model (68.3% vs. 63.4%), the P1 set contributes only about one demonstration per task on average, limiting its impact. Finally, by introducing environment-level augmentation (E_{Aug}), the supplemented pool becomes a more diverse dataset. The union of dual-level augmentation and Base model generated rollouts (P1 + P2 + P5) then achieves the best performance of **69.6%**, underscoring the effectiveness of our proposed dual-level augmentation.

b) Effects of failure rollouts: In Table IV, we evaluate the importance of the success filter by analyzing the effect of introducing near-success (NS) and failure demonstrations. Incorporating NS samples results in only minor degradation (*i.e.*, 3.5% in the fourth evolution round), as the model still learns predominantly correct actions. In contrast, injecting failure demonstrations causes significant performance drops (over 10% in both the first and fourth rounds), since imitation learning leads the policy to replicate incorrect actions.

c) Study of various initialization: We investigate the impact of different model initializations on final performance through Table II. SEIL provides a framework for self-evolution of any model. The experimental results show that stronger models generate more diverse rollout pools, and the models trained on these rollouts exhibit greater performance improvement. For instance, a one-shot BAKU model trained on an 8-shot BAKU generated demonstration set achieved a 26.3% performance increase.

3) Effect of the Attempt Numbers. Table VI studies the effect of the number of recorded demonstrations (RD) per round. We apply E_{Aug} to the base model and evolve using all generated demonstrations without selection, incrementally expanding the pool. With only 10 rollouts per round, evolution is ineffective, yielding marginal gains (*e.g.*, +0.2% from round 2 to 3). In contrast, 50 or 100 rollouts lead to early saturation, and in the 100-rollout setting performance even drops to 71.7% in round 2, suggesting overfitting. Collecting too many demonstrations per round causes the model to over-adapt to current RD instead of gradually refining it. Overall, 20 rollouts per round achieves the best performance, peaking at 74.2% in round 5.

TABLE VII
ABLATION STUDY ON SELECTION SCHEME. MIXED REPRESENT HALF DEMONSTRATIONS IN DESCENDING ORDER AND THE REST HALF IN ASCENDING ORDER.

Scheme	Uniform	Descending	Ascending	Mixed
SR	70.8%	71.0%	73.7%	72.2%

4) Effect of the Selecting Demonstration Pool Size. Table V reveals a clear trade-off: enlarging the candidate rollout pool while fixing the number of selected demonstrations improves performance by enabling higher-quality selection, but increases computational cost. Since selection is performed on successful rollouts, obtaining k successful trajectories requires approximately k /SR simulator attempts in expectation. Consequently, a larger pool proportionally increases simulation time, particularly under low-SR regimes, leading to a compute-quality trade-off between data curation and rollout efficiency.

5) Effect of the Selector. *a) Selected rollouts visualization:* In Fig. 4, we analyze the inference results of the selector across different rollouts. Through the learning of classification tasks, the selector can differentiate between the expert demonstration and the model's outputs based on class confidence. This enables the effective identification of diversity at both the trajectory level and the state level.

b) Effect of the accuracy: The last row of the Table II reports the rollout selection success rate. In the 8-shot setting, models trained with only eight demonstrations already achieve high rollout success rates, reaching 80% for rollouts generated by 8-shot BAKU and 70% for those generated by MT-ACT. In contrast, the lower success rate observed in the 1-shot setting is attributed to the limited generalization capability of the 1-shot model initialization, which can only succeed in scenarios highly similar to the demonstration; consequently, selecting from inherently less diverse rollouts leads to a lower overlap with human preferences.

c) Effect of the selection scheme: Table VII reports success rates under different selection strategies. From a 50-attempt demonstration pool generated by Base and M_{Aug} , we select 20 samples per task. All selector-based methods outperform random sampling. Selecting demonstrations with the lowest confidence achieves the best performance (73.7%), suggesting that prioritizing diverse samples improves policy learning. The Mixed strategy also surpasses the Descending strategy (72.2% vs. 71.0%), further highlighting the advantage of diversity over selecting only low-confidence demonstrations.

d) Effect of the trajectory classification task: Table VIII reports selector accuracy on the validation set. Incorporating images improves classification by an average of 7%, indicating

TABLE VIII
ACCURACY OF PRETRAINED CLASSIFICATION TASK

Shot\Input	Sequence	Img+Sequence
1 shot	53.0%	57.0%
2 shot	56.0%	61.0%
4 shot	54.5%	63.5%
8 shot	74.5%	85.0%

more precise trajectory recognition. Nevertheless, the selector remains competitive without images, suggesting that trajectories inherently encode task-specific attributes and scene information.

VI. LIMITATIONS AND FUTURE WORK

Despite SEIL's effectiveness in few-shot settings, several limitations remain. First, the framework relies on a simulator to generate demonstrations, which may not be accessible in real-world scenarios. However, compared to the tedious process of manual data collection, our method offers an alternative that can efficiently scale datasets in a more automated and cost-effective manner. Second, SEIL requires multi-stage training, which introduces additional evolving time compared to one-stage approaches.

As a future direction, we view our environmental augmentation as a form of positional OOD variation, as it introduces changes in spatial configurations during training. This observation suggests that SEIL may provide a useful foundation for broader OOD exploration in the future, including more diverse environment shifts, unseen layouts, and other distribution changes encountered in real-world scenarios.

VII. CONCLUSION

In this letter, we proposed Self-Evolved Imitation Learning, a novel framework that progressively evolves a weak few-shot policy through interactions with a simulator. SEIL introduces a dual-level augmentation strategy to generate diverse demonstrations and a lightweight trajectory selector to identify informative samples for efficient policy refinement. Extensive experiments across multiple benchmarks validate the effectiveness of SEIL and demonstrate the individual contributions of its core components.

REFERENCES

- [1] S. Haldar, Z. Peng, and L. Pinto, "BAKU: An efficient transformer for multi-task policy learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 141208–141239.
- [2] C. Chi et al., "Diffusion policy: Visuomotor policy learning via action diffusion," *Int. J. Robot. Res.*, vol. 44, no. 10–11, pp. 1684–1704, 2023.
- [3] P. Florence et al., "Implicit behavioral cloning," in *Proc. Conf. Robot Learn.*, 2022, pp. 158–168.
- [4] N. M. Shafiullah, Z. Cui, A. A. Altanzaya, and L. Pinto, "Behavior transformers: Cloning k modes with one stone," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 22955–22968.
- [5] A. Brohan et al., "RT-1: Robotics transformer for real-world control at scale," in *Proc. Robot.: Sci. Syst. Conf.*, 2023.
- [6] B. Liu et al., "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 44776–44791.
- [7] J. N. K. Wah, "Revolutionizing surgery: AI and robotics for precision, risk reduction, and innovation," *J. Robot. Surg.*, vol. 19, no. 1, Jan. 2025, Art. no. 47.
- [8] D. Silver et al., "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016.
- [9] J. Ibarz et al., "How to train your robot with deep reinforcement learning: Lessons we have learned," *Int. J. Robot. Res.*, vol. 40, pp. 698–721, 2021.
- [10] A. Li, B. Boots, and C.-A. Cheng, "MAHALO: Unifying offline reinforcement learning and imitation learning from observations," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 19360–19384.
- [11] P. Rashidinejad, B. Zhu, C. Ma, J. Jiao, and S. Russell, "Bridging offline reinforcement learning and imitation learning: A tale of pessimism," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 11702–11716.
- [12] S. Dey, T. Marzullo, X. Zhang, and G. Henze, "Reinforcement learning building control approach harnessing imitation learning," *Energy AI*, vol. 14, 2023, Art. no. 100255.
- [13] G. Tiboni, K. Arndt, and V. Kyrki, "DropO: Sim-to-real transfer with offline domain randomization," *Robot. Auton. Syst.*, vol. 166, 2023, Art. no. 104432.
- [14] A. Piche, R. Pardinaz, D. Vazquez, I. Mordatch, and C. Pal, "Implicit offline reinforcement learning via supervised learning," 2022, *arXiv:2210.12272*.
- [15] T. Z. Zhao et al., "Offline meta-reinforcement learning for industrial insertion," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 6386–6393.
- [16] D. Han, B. Mulyana, V. Stankovic, and S. Cheng, "A survey on deep reinforcement learning algorithms for robotic manipulation," *Sensors*, vol. 23, no. 7, Apr. 2023, Art. no. 3762.
- [17] K. Arulkumar, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
- [18] G.-H. Kim et al., "DemoDice: Offline imitation learning with supplementary imperfect demonstrations," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [19] P. F. Christiano et al., "Deep reinforcement learning from human preferences," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30.
- [20] R. Agarwal, D. Schuurmans, and M. Norouzi, "An optimistic perspective on offline reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 104–114.
- [21] A. Mandlekar et al., "What matters in learning from offline human demonstrations for robot manipulation," in *Proc. Conf. Robot Learn.*, 2021.
- [22] A. Mete, H. Xue, A. Wilcox, Y. Chen, and A. Garg, "QUEST: Self-supervised skill abstractions for learning continuous control," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 4062–4089.
- [23] H. Chen, B. Sun, A. Zhang, M. Pollefeys, and S. Leutenegger, "VidBot: Learning generalizable 3D actions from in-the-wild 2D human videos for zero-shot robotic manipulation," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2025, pp. 27661–27672.
- [24] A. Stone et al., "Open-world object manipulation using pre-trained vision-language models," in *Proc. 7th Conf. Robot Learn.*, 2023, pp. 1–21.
- [25] T. Yu et al., "Scaling robot learning with semantically imagined experience," in *Proc. Robot.: Sci. Syst. Conf.*, 2023.
- [26] F. Lin et al., "Data scaling laws in imitation learning for robotic manipulation," in *Proc. Int. Conf. Learn. Representations*, 2025.
- [27] T. Yu, Z. Lu, X. Jin, Z. Chen, and X. Wang, "Task residual for tuning vision-language models," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2023, pp. 10899–10909.
- [28] X. Li, D. Lian, Z. Lu, J. Bai, Z. Chen, and X. Wang, "GraphAdapter: Tuning vision-language models with dual knowledge graph," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 13448–13466, 2023.
- [29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, Jun. 2016, pp. 770–778.
- [30] H. Bharadhwaj, J. Vakil, M. Sharma, A. Gupta, S. Tulsiani, and V. Kumar, "RoboAgent: Generalization and efficiency in robot manipulation via semantic augmentations and action chunking," *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 4788–4795. [Online]. Available: <https://api.semanticscholar.org/CorpusID:261518421>
- [31] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proc. Robot.: Sci. Syst. Conf.*, 2023.